Introduction to Paging Concepts

Paging is a memory management technique that removes the need for contiguous memory allocation.
It divides **virtual memory** into blocks called **pages**, and **physical memory (RAM)** into blocks called **frames**.

This method helps the operating system:

- Use memory more efficiently
- Simplify memory allocation for processes

How Paging Works

1. **Virtual Memory:**
   Each process has its own virtual address space, divided into fixed-size units called *pages*.

2. **Physical Memory:**
   Physical memory (RAM) is divided into *frames*, each of the same size as a page.

3. **Page Table:**
   The operating system maintains a *page table* for every process.
   This table maps virtual pages to physical frames.

4. **Page Fault:**
   When a process tries to access a page not currently in RAM, a *page fault* occurs.
   The OS then loads the required page from disk into memory.

5. **Page Replacement:**
   If all frames are full, the OS must replace one of the existing pages.
   It uses **page replacement algorithms** such as:

   - **FIFO (First In, First Out)**
   - **LRU (Least Recently Used)**
   - **Optimal Replacement**

Tasks

Task 1: Page Table Simulation

Objective:
Simulate how a page table works and demonstrate how virtual addresses are translated into physical addresses.

Instructions:

1. Choose a set of **virtual memory addresses** and their **page numbers**.
2. Define a **page table** with at least **5 entries** (map virtual pages to physical frames).
3. Write a small **program or spreadsheet** to:
    - Display the page table
    - Translate virtual addresses into physical addresses using the page table
    - Handle a **page fault** by randomly replacing a page in the table

Deliverables:

- Code file or spreadsheet
- A short report explaining:
    - The steps you followed
    - The results of the translation process

Task 2: Page Replacement Algorithm Analysis

Objective:
Analyze and compare how different page replacement algorithms perform.

Instructions:

1. Select **at least three algorithms** — for example:
    - LRU (Least Recently Used)
    - FIFO (First In, First Out)
    - Optimal Replacement
2. Use a **reference string of page requests** to simulate memory allocation for a given number of frames (e.g., 3 frames).
3. For each algorithm:
    - Count the number of **page faults**
    - Create a **graph or table** comparing results
4. Write a **brief analysis** discussing:
    - Strengths and weaknesses of each algorithm
    - Which algorithm performs best for your test case

Deliverables:

- Simulation results and visual comparison (graph/table)
- A short report summarizing your analysis

Conclusion

These two tasks will help you:

- Understand how **paging and memory management** work in operating systems
- Gain **hands-on experience** with memory allocation and page replacement strategies

Good luck — and have fun exploring how operating systems manage memory behind the scenes!